Propliq V2 API Documentation

This documentation covers the implementation of **Booking**, **Calendar Management**, and **Live AI Chat WebSockets** for both Web (Next.js) and Mobile (Flutter).

---

1. Booking API

Guests use this API to reserve properties (Apartments, Hotels, or Shortlets).

> ***Optimization Note:*** *Bookings are processed against a real-time availability engine. For high-traffic events, ensure your UI handles the* `409 Conflict` *status gracefully when rooms sell out instantly.*

How it works:

When a guest attempts to book, the system performs a real-time availability check against both **confirmed bookings** and **manual host blocks**. If the dates are free, a `pending` booking is created, and a payment URL is returned for the guest to complete the transaction.

Create a Booking

Endpoint: `POST /api/v1/bookings`

Authentication: Required (Bearer Token)

Request Body:

```
{
    "property_id": 1,
    "check_in_date": "2026-04-10",
    "check_out_date": "2026-04-15"
}
```

Response (Success - 201):

```
{
    "success": true,
    "data": {
        "booking": {
            "uuid": "...",
            "booking_code": "PLQ-ABC123",
            "amount": 50000.00,
            "status": "pending"
        },
        "payment": {
            "authorization_url": "https://checkout.paystack.com/..."
        }
    }
}
```

---

2. Calendar Management (Agent/Host)

Agents can manage room availability by manually blocking or unblocking dates.

Why use this?

Hosts may need to take a room off the market for maintenance, personal use, or offline bookings. By "blocking" a date, the host ensures that no guest can book the property through the app for those specific days.

Get Room Calendar (Agent)

Endpoint: `GET /api/v1/agent/calendar/room/{room_id}?start_date=2026-04-01&end_date=2026-04-30`

Block Dates (Agent)

Endpoint: `POST /api/v1/agent/calendar/room/{room_id}/block`

Request Body:

```
{
    "start_date": "2026-04-20",
    "end_date": "2026-04-22",
    "reason": "Maintenance"
}
```

Unblock Dates (Agent)

Endpoint: `DELETE /api/v1/agent/calendar/unblock/{uuid}`

---

3. Public Availability Check

Guests can check if a room is available before booking.

Endpoint: `GET /api/v1/rooms/{room_id}/availability?start_date=2026-04-01&end_date=2026-04-30`

Response:

```
{
    "success": true,
    "data": [
        { "date": "2026-04-01", "status": "available", "is_blocked": false },
        { "date": "2026-04-02", "status": "booked", "is_blocked": true },
        { "date": "2026-04-03", "status": "blocked", "is_blocked": true, "metadata": { "reason": "Maintenance" } }
    ]
}
```

---

4. AI Chat WebSockets (Live Updates)

The AI Chat uses **Laravel Reverb** (Pusher-compatible) for real-time response streaming.

Why WebSockets?

Instead of making the user wait for a standard HTTP response (which can take 5-10 seconds for AI), the app can immediately acknowledge the query and "listen" for the AI's response to be pushed live. This provides a much smoother, chat-like experience.

Next.js Implementation

1. **Install:** `npm install laravel-echo pusher-js`

> ***Security Note:*** *Anonymous AI conversations are restricted by a signed token or user session logic. Ensure the* `conversationId` *is stored securely in the client state and never exposed in public logs.*

2. **Setup:**

```
import Echo from 'laravel-echo';
import Pusher from 'pusher-js';

const echo = new Echo({
    broadcaster: 'reverb',
    key: 'YOUR_REVERB_KEY',
    wsHost: 'api.proplinq.com',
    wsPort: 8080,
    forceTLS: false,
    enabledTransports: ['ws', 'wss'],
});

// Subscribe to conversation
echo.channel(`ai-chat.${conversationId}`)
    .listen('.ai.chat.response', (data) => {
        console.log("AI Message:", data.response);
    });
```

Flutter Implementation

1. **Dependency:** `pusher_channels_flutter: ^2.2.0`
2. **Setup:**

```
import 'package:pusher_channels_flutter/pusher_channels_flutter.dart';

PusherChannelsFlutter pusher = PusherChannelsFlutter.getInstance();

await pusher.init(
  apiKey: "YOUR_REVERB_KEY",
  host: "api.proplinq.com",
  port: 8080,
  onEvent: (event) {
    if (event.eventName == 'ai.chat.response') {
      print("AI Response: ${event.data}");
    }
  },
);

await pusher.subscribe(channelName: "ai-chat.$conversationId");
await pusher.connect();
```

Developer Notes

- **Free WebSocket:** We use Laravel Reverb, which is self-hosted and free (no Pusher fees).
- **Manual Availability:** If `availability_status` is `false` for a room, it means the agent hasn't set any manual availability data yet.

- **AI Engine:** All AI responses are processed in a dedicated Docker container (`ai-engine`) and broadcasted immediately to the user's channel.

Structural & Response Standard

- **Unified Responses**: All API endpoints follow a strict format:

```
{
    "status": true,
    "message": "Success message",
    "data": { ... }
}
```

- **Error Handling**: Standardized `4xx` and `5xx` error responses are returned using the same structure with `status: false`.
- **WebSocket Scalability**: Reverb is configured for easy horizontal scaling using a Redis backplane if traffic increases.